



Chapter 4

JavaScript

College of Information and Communication Technology

Can Tho University

Dr. Trieu Thanh Ngoan

Content

1. Introduction
2. Using JavaScript in an HTML documents
3. JS language basics
4. JS functions
5. DOM
6. OOP in JS

Content

1. Introduction
2. Using JavaScript in an HTML documents
3. JS language basics
4. JS functions
5. DOM
6. OOP in JS

◆ What is JavaScript?

- JavaScript is a programming language (no relation to Java, it is named just for marketing reason 😊)
- Currently is the only scripting language for browsers
- It's supported by all popular browsers: Firefox, Chrome, Safari, etc.
- JavaScript is used by billions of websites to create functions, check data, exchange information with web servers, etc.
- JavaScript is an object-oriented, interpreted programming language that is executed by the browser on the client side.

◆ What is JavaScript?

History:

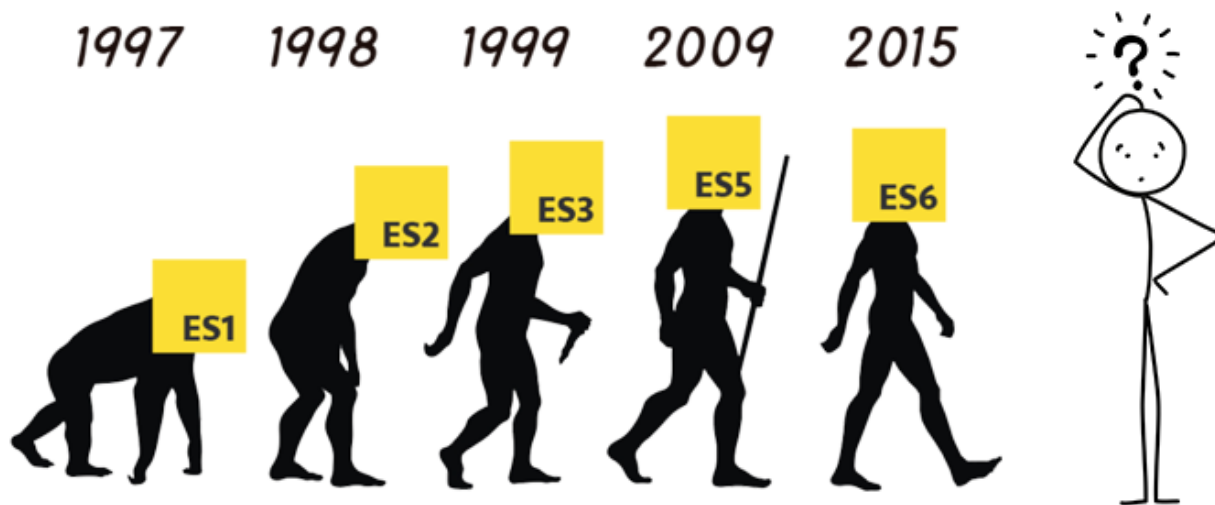
- Originally developed by Brendan Eich at Netscape (named LiveScript at that time)
- Became a joint venture of Netscape and Sun in 1995 (renamed JavaScript)
- Now standardized by the ECMA (ECMA-262, also ISO 16262)

ECMA (European Computer Manufacturers Association) is an industry association dedicated to the standardization of information and communication systems

◆ What is JavaScript?

ECMAScript (ES): a standard for scripting languages

JavaScript: a programming language based on ECMAScript (the most popular implementation of ES)



Why JavaScript?



+



produces



Describes the
content and
structure of the page

Describes the
appearance and
style of the page

A web page... that doesn't
do anything, except:

- Shows the information
- Links to other webpages

If we want dynamic, more friendly and interactive webpages (automation, animation, interactivity, etc.)

JavaScript comes to help!

What JS Can Do?

User **interaction** through form are easy

Document Object Model (DOM) makes it possible to create **dynamic HTML** with JS:

- can change the HTML page content
- can show/hide HTML page elements
- can change the HTML element's attributes
- etc.

And much **more features** that we can do with **event-driven** computation...

◆ Basic Features of JS

An **interpreter** language (no compilation by developers, it is compiled and executed on-the-fly by browser)

Dynamic typed (no need to declare variables, auto type-casting)

Supports the OOP approach (but lack of some basic features of OOP such as polymorphism)

Content

1. Introduction
2. Using JavaScript in an HTML documents
3. JS language basics
4. JS functions
5. DOM
6. OOP in JS

❖ Internal JS Code

JS code can be embedded directly in an HTML page using `<script>` tag:

```
<script>
```

```
... JavaScript code ...
```

```
</script>
```

There may be a number of scripts in an HTML page

Scripts can be placed in the `<head>` or in the `<body>` section

❖ **Note:** Placing scripts at the **bottom** of the `<body>` element **improves the display speed**, because script compilation slows down the display

◆ External JS Code

JS script (code) can also be placed in **external files** and linked to HTML pages

JS script files have the extension **.js**

To link a JS script, use the **<script>** tag with the **src** attribute:

```
<script src="<script ULR>"></script>
```

External JS script **advantages**:

- **Separate** JS code and HTML code (easy to read and maintain)
- JS code can be **reused/shared** in HTML pages
- **Cached** JS files can **speed-up** page load, and much more...

Recommended!

❖ External JS Code

External scripts can be referenced with a full URL (placed in other sites or the same site) or with a relative path (placed in the same site)

Example:

```
<script src="scripts/myscript1.js"></script>
```

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
```

❖ Notes:

- External JS scripts cannot contain `<script>` tags
- The `<script>` tag can be placed **anywhere** in the HTML page
- An HTML page can use **multiple script files**: use multiple `<script>` tags to link to multiple script files

◆ How Does JS Code Get Loaded?

If a browser receive an HTML page with a reference to a JS file:

- 1) It will **make a request** to the web server for the **script file**
- 2) After receiving the script files, it will **execute** the script file

JS script execution:

There is **no main function** as other programming languages

⇒ The script code is executed from the **top to the bottom**

JS functions in the script file are only executed when called

◆ Output of JS

JS can display data in different ways:

- Writing into an HTML element, using `innerHTML` property
- Writing into HTML page, using `document.write()`
(**Note:** *Using `document.write()` after an HTML document is loaded, will delete all existing HTML*)
- Writing into an alert box, using `window.alert()`
- Output to browser console, using `console.log()`
(*usually used for debugging purpose*)

◆ Output of JS

Writing to an HTML element:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Today is: <span id="curdate"></span></h1>
    <p>5 + 6 = <span id="calc"></span></p>
    <script>
      var d = (new Date()).toString().substring(0, 15);
      document.getElementById("curdate").innerHTML = d;
      document.getElementById("calc").innerHTML = 5 + 6;
    </script>
  </body>
</html>
```

Today is: Mon Feb 18 2019

5 + 6 = 11

◆ Output of JS

Writing into an alert box:

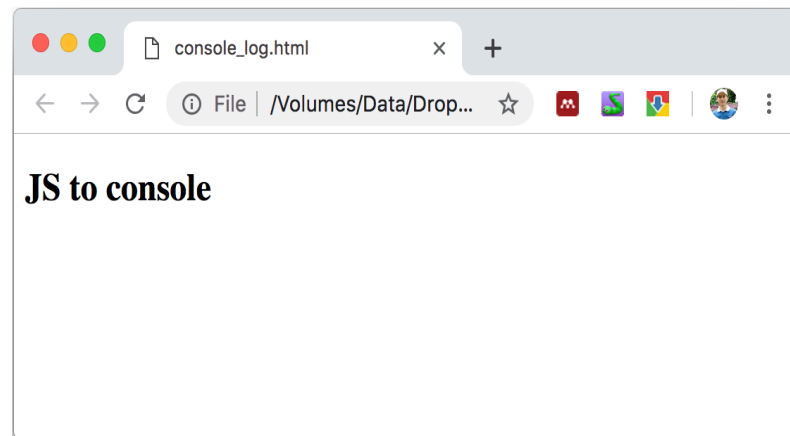
```
<!DOCTYPE html>
<html>
  <body>
    <h2>JS Output</h2>
    <button onclick="alert('Hello!')">Click me to say Hello</button>
  </body>
</html>
```



◆ Output of JS

Output to browser console:

```
<!DOCTYPE html>
<html>
  <body>
    <h2>JS to console</h2>
    <script>
      console.log('Hello, world!');
    </script>
  </body>
</html>
```



Output of JS

Output to browser console (cont.):

The diagram illustrates the process of viewing JavaScript output in the browser console. It features two browser window screenshots. The first screenshot shows a right-click context menu on the page content, with a yellow box labeled '1. Right click' pointing to the menu. The second screenshot shows the 'Inspect' option selected in the menu, with a yellow box labeled '2. Click Inspect' pointing to it. The third screenshot shows the 'Console' tab selected in the browser's developer tools, with a yellow box labeled '3. Choose Console' pointing to it. A blue starburst graphic with the text 'for debugging' is positioned above the second screenshot. The console output shows 'Hello, world!' and the source 'console_log.html:6'.

JS to console

1. Right click

2. Click Inspect

3. Choose Console

for debugging

JS to console

Hello, world! console_log.html:6

Content

1. Introduction
2. Using JavaScript in an HTML documents
- 3. JS language basics**
4. JS functions
5. DOM
6. OOP in JS

◆ Statements

- Script/Program = { set of instructions/statements }
- A **statement** is composed of: values (constant/literals or variable), operators, expressions, keywords, and comments
- A statement **ends with a semicolon ;**
- Multiple **while-space** are ignored (so we can add as much as white-space for readability)
- A statement can be **broken** into multiple lines
- A set of statements can be grouped in **code block**, using { ... }

◆ Comments

For code readability and maintenance

Comments will be **ignored** by JS (doesn't interpret and execute)

Single line comments:

- Start with //
- ... to the end of line

Multiline comments:

- Start with /*
- End with */

Good practice to use comments as much as possible

◆ Variables

Declaration: `var/let/const` <var_name> [= <expression>];

Example: `let/const name = 'Jerry', age = 40;`

Datatype is not necessary in the variable declaration (however, JS does has datatypes: number, boolean, string, object, function, undefined, Array, Object, Date)

JS variables can also be used without declaration

Variable **naming rules** (same as identifier naming rules):

- Names can **contain** letters, digits, underscores, and dollar signs
- Names must **begin** with a letter, \$ and _
- Names are **case sensitive**

◆ Variables

Declaring a variable **without assigning value**: the variable will have the value **undefined**

```
var age; //  $\Rightarrow$  age = undefined
```

Re-declaring a variable doesn't erase its value

```
var myAge = 40;
```

```
var myAge; //  $\Rightarrow$  age still = 40
```

Get the **datatype** of a variable: use the **typeof** operator

Example: **typeof myAge** returns a string “**number**”

We can **declare multiple variables** in one statement (using comma to separate variable names)

Variables

When to Use var, let, or const?

1. Always declare variables
2. Always use const if the type should not be changed (Arrays and Objects)
3. Only use let if you cannot use const
4. Never use var if you can use let or const.

	Scope	Redeclare	Reassign	Hoisted
var	No	Yes	Yes	Yes
let	Yes	No	Yes	No
const	Yes	No	No	No

Hoisting is moving all declarations to the top of the current scope

Variables

```
function showScope() {
  if (true) { var functionScopedVar = "I'm visible everywhere in this function"; }
  console.log(functionScopedVar); // Output: I'm visible everywhere in this function
}
showScope();
console.log(functionScopedVar); // ReferenceError: functionScopedVar is not defined
function showBlockScope() {
  if (true) {
    let blockScopedLet = "I'm only visible inside the if block";
    const blockScopedConst = "Me too, only inside the if block";
    console.log(blockScopedLet); // Output: I'm only visible inside the if block
    console.log(blockScopedConst); // Output: Me too, only inside the if block
  }
  console.log(blockScopedLet); // ReferenceError: blockScopedLet is not defined
  // console.log(blockScopedConst); // ReferenceError: blockScopedConst is not defined
}
showBlockScope();
```

◆ Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (<u>ES2016</u>)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Arithmetic

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

Assignment

Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Comparison

Operator	Description
&&	logical and
	logical or
!	logical not

Logical

Operator	Description
+	Addition

String

JS also provides **bitwise operators** such as & (AND), | (OR), ~ (NOT)

◆ Datatypes

JS variables don't have types, but the values do

Primitive datatypes:

- **Boolean:** true or false
- **Number:** everything is double (no integers)
- **String:** text enclosed in 'single' or "double" quotes
- **Null:** `null`, a value meaning “this has no value” (null is an object)
- **Undefined:** `undefined`, i.e. “not assigned”

Object types:

- **Array, Date, Object, ...**

◆ Number

All numbers in JS are **real numbers**, no integer

Operators are like C/C++ or Java

Precedence like C/C++ or Java

Some **special values**:

NaN (not-a-number)

+**Infinity**

-Infinity



```
NaN == NaN: false
NaN != NaN: true
Number.isNaN(value)
```

There is a **Math** class, which provide basic math functions (such as **Math.ceil()**, **Math.floor()**, etc.)

◆ String

Can be defined with **single quote** (preferred) or **double quote**

Immutable

No char type: letters are strings with length 1

Methods: `charAt`, `charCodeAt`, `fromCharCode`, `indexOf`, `lastIndexOf`, `replace`, `split`, `substring`, `toLowerCase`, `toUpperCase`

Property: `length`

The only operator: `+`

```
var s = "Lion Messi";  
var fname = s.substring(0, s.indexOf(" "));  
var len = s.length;  
var s2 = 'Cristian Ronaldo';
```

◆ String and Number Conversion

Converse between string and number:

```
let count = 10;
let s1 = "" + count; //"10"
let s2 = count + " student"; //"10 student"
let n1 = parseInt("10 student"); //10
let n2 = parseFloat("blablabla"); //NaN
let s = count.toString(); //"10"
let n3 = Number(true); //1
let n4 = Number(""); //0
```

Get a character in a string at a specified position:

```
let s = "Hello World";
let firstLetter = s[0]; //"H"
let firstLetter = s.charAt(0); //"H"
```


◆ Boolean

Literal values: **true** and **false**

Operators: **&&**, **||** and **!**

Non-boolean values can be used as a boolean value with the following **type coercion** (implicitly **conversion**):

null, undefined, 0, NaN, "", and "" are evaluated to **false**

Everything else is evaluated to **true**

We can also explicitly convert an arbitrary value to boolean using the **Boolean()** constructor or **!!** operator

◆ Boolean

```
let complete = true;
let age = 40;
let old = age > 60; //false
let boolValue = Boolean("to ambiguous"); //true
let boolValue1 = !!(""); //false

let username = "...";

if (username) {
    // username is defined
}
else {
    //username is not defined
}
```

◆ Null and Undefined

`null` is a value representing the **absence of a value** (similar to `null` value in Java and `nullptr` in C++)

`undefined` is the value given to a variable that **has not had a value**

However, we can also set a variable's value to `undefined` 😞

```
let x = null;  
let y;  
console.log(x);  
console.log(y);
```

```
let x = null;  
let y = undefined;  
console.log(x);  
console.log(y);
```

null
undefined
>

◆ Equality

Loosely comparison

JavaScript operators `==` and `!=` are basically broken: they do an **implicit type conversion** before the comparison

```
' ' == '0' // false (two different strings)
' ' == 0   // true (' ' ~ 0, 0 == 0 is true)
0 == '0'   // true (0 ~ '0' => '0' == '0' is true)
NaN == NaN // false (JS rule ☺)
[' '] == ' ' // true
false == undefined // false
false == null      // false
null == undefined  // true
```

Equality

Strict comparison

ECMAScript standard added `===` and `!==` operators with the better behavior

```
' ' === '0' // false
' ' === 0 // false
0 === '0' // false
NaN === NaN // still weirdly false
[''] === '' // false
false === undefined // false
false === null // false
null === undefined // false
```

```
' ' == '0' // false
' ' == 0 // true
0 == '0' // true
NaN == NaN // false
[''] == '' // true
false == undefined // false
false == null // false
null == undefined // true
```

```
null == undefined: true
null === undefined: false
```

◆ Array

An Object type used to store **multiple values** in a **single variable**

0-based indexing

Mutable

Can check size via **length** property

```
const list = []; // Creates an empty array
```

```
//create a pre-filled array
```

```
const groceries = ['milk', 'cocoa puffs'];
```

```
groceries[1] = 'kix'; //access an element of the array
```

◆ Arrays

An array can be used as a list, queue, stack, etc.

Basic methods:

- **push/pop**: add/remove the element at the end, return an element
- **shift/unshift**: remove/add an element at the head, return an element
- **concat**: joints two or more arrays, return the jointed array
- **splice**: add/remove elements from an array, return the array after adding/removing
- **sort/reserve**: sort/reverse the order of elements in the array, return the result after sorted, reserved
- **slice**: select a part of an array, return the selected part

https://www.w3schools.com/jsref/jsref_obj_array.asp

◆ Arrays

```
const a = ["Truc", "Lan"];  
a.sort();           //a=["Lan", "Truc"]  
a.push("Cuc");      //a=["Lan", "Truc", "Cuc"]  
a.unshift("Mai");   //a=["Mai", "Lan", "Truc", "Cuc"]  
  
const b = a.slice(1, 3); // b=["Lan", "Truc"]  
a.splice(2, 1, "Tung", "Dao"); //a=["Mai", "Lan", "Tung", "Dao", "Cuc"]  
a.pop();           //a=["Mai", "Lan", "Tung", "Dao"]  
a.shift();         //a=["Lan", "Tung", "Dao"]  
  
let i = a.indexOf("Tung"); //1
```


◆ Array Iteration Methods

Simple for loop (usually combined with the length property)

for ... of statement

Array.forEach():

calls a **callback function** once for each array element

The callback function takes **3 parameters**: item **value**, the **index** and the **array** itself

```
let iterable = [10, 20, 30];
for (const value of
iterable) {
  console.log(value);
}
```

10
20
30

```
let iterable = [10, 20, 30];
for (let value of iterable)
{
  value += 1;
  console.log(value);
}
```

11
21
31

```
const numbers = [45, 4, 9, 16, 25];
let txt = "";
numbers.forEach(myFunction);

function myFunction(value, index, array) {
  txt += value + "<br>";
}
```

◆ Date

The Date class is used to manipulate date and time data types.

Create Date:

```
let d = new Date();  
let d = new Date(milliseconds);  
let d = new Date(dateString);  
let d = new Date(year, month, day, hours, minutes, seconds,  
milliseconds);
```

Methods:

```
getDate(), getDay(), getMonth(), getFullYear()  
getHours(), getMinutes(), getSeconds(), getMilliseconds()  
setDate(), setMonth(), setFullYear()  
setHours(), setMinutes(), setSeconds(), setMilliseconds()
```

Date object reference: http://w3schools.com/jsref/jsref_obj_date.asp

◆ Date

Example:

```
let today = new Date()  
let d1 = new Date("October 13, 1975 11:13:00")  
let d2 = new Date(79,5,24)  
let d3 = new Date(79,5,24,11,33,0)
```

```
let myDate = new Date();  
myDate.setFullYear(2010,0,14);
```

```
let myDate1 = new Date(myDate);  
myDate1.setDate(myDate1.getDate()+5)
```

◆ Date

Example:

```
let x = new Date();
x.setFullYear (2100, 0, 14);
let today = new Date();

if (x > today) {
    alert("Today is before 14th January 2100");
}else{
    alert("Today is after 14th January 2100");
}
```

◆ RegExp Type

Regular expression:

A sequence of characters that forms a **search pattern**

Regex is a common shorthand for a regular expression

JavaScript RegExp is an Object for handling Regular Expressions

Syntax

```
let txt=new RegExp(pattern,modifiers); or
```

```
let txt=/pattern/modifiers;
```

Pattern: defines the pattern of the expression.

Modifiers: specifies whether the search should be global and case-sensitive.

◆ Modifiers

g - Performs a global match (find all)

i - Performs case-insensitive matching

m – Performs Multiline mode matching

Example:

```
// Match all instances of "at" in a string.  
var pattern1 = /at/g;  
  
//Match the first instance of "bat" or "cat",  
//regardless of case  
var pattern2 = /[bc]at/i;
```

◆ RegExp Object

Brackets ([]): characters enclosed in square brackets [].

Expression	Description
<code>[abc]</code>	Find any character between the brackets
<code>[^abc]</code>	Find any character not between the brackets
<code>[0-9]</code>	Find any digit from 0 to 9
<code>[A-Z]</code>	Find any character from uppercase A to uppercase Z
<code>[a-z]</code>	Find any character from lowercase a to lowercase z
<code>[A-z]</code>	Find any character from uppercase A to lowercase z
<code>[adgk]</code>	Find any character in the given set
<code>[^adgk]</code>	Find any character outside the given set
<code>(red blue green)</code>	Find any of the alternatives specified

◆ RegExp Object

Metacharacter: characters with a special meaning.

Metacharacter	Description
<code>.</code>	Find a single character, except newline or line terminator
<code>\w</code>	Find a word character
<code>\W</code>	Find a non-word character
<code>\d</code>	Find a digit
<code>\D</code>	Find a non-digit character
<code>\s</code>	Find a whitespace character
<code>\S</code>	Find a non-whitespace character
<code>\b</code>	Find a match at the beginning/end of a word
<code>\B</code>	Find a match not at the beginning/end of a word
<code>\0</code>	Find a NUL character
<code>\n</code>	Find a new line character
<code>\f</code>	Find a form feed character
<code>\r</code>	Find a carriage return character
<code>\t</code>	Find a tab character
<code>\v</code>	Find a vertical tab character
<code>\xxx</code>	Find the character specified by an octal number xxx
<code>\xdd</code>	Find the character specified by a hexadecimal number dd
<code>\uxxxx</code>	Find the Unicode character specified by a hexadecimal number xxxxx

◆ RegExp Object

Quantifiers: define the numbers of characters or expressions to match.

Quantifier	Description
<u>n^+</u>	Matches any string that contains at least one n
<u>n^*</u>	Matches any string that contains zero or more occurrences of n
<u>$n^?$</u>	Matches any string that contains zero or one occurrences of n
<u>$n\{X\}$</u>	Matches any string that contains a sequence of X n 's
<u>$n\{X,Y\}$</u>	Matches any string that contains a sequence of X to Y n 's
<u>$n\{X, \}$</u>	Matches any string that contains a sequence of at least X n 's
<u>$n\\$</u>	Matches any string with n at the end of it
<u>n</u>	Matches any string with n at the beginning of it
<u>$?=n$</u>	Matches any string that is followed by a specific string n
<u>$?!n$</u>	Matches any string that is not followed by a specific string n

◆ RegExp Object

Common example:

Email Address: `\b[A-z0-9._%+~]+@[A-z0-9.-]+\.[A-z]{2,6}\b`

Date String: `dd/mm/yyyy`

`(0[1-9]|[12][0-9]|3[01])(- /.)(0[1-9]|1[012])(- /.)(19|20)[0-9]{2}`

International phone number: `^\+([0-9] ?){6,14}[0-9]$`

Integer: `\b\d+\b`

Text with letters and digits:

`^[A-Z0-9]+$`

Text with length from 1 to 10:

`^[A-Z]{1,10}$`

◆ RegExp Object

Example:

```
<html>
<body>

<script type="text/javascript">
var str = "Visit W3Schools";
var patt1 = /w3schools/i;
document.write(str.match(patt1));
</script>

</body>
</html>
```

W3Schools

```
<html>
<body>

<script type="text/javascript">
var patt1=new RegExp("e");

document.write(patt1.test("The
best things in life are free"));
</script>

</body>
</html>
```

true

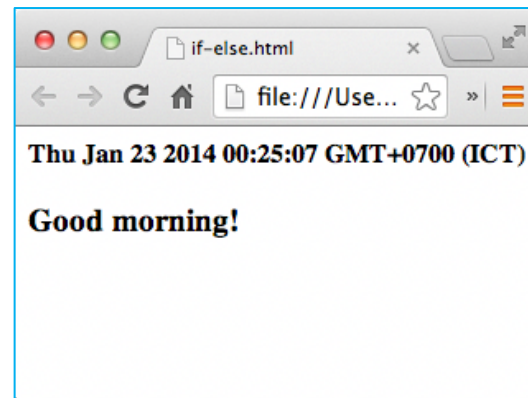
◆ Condition Statements – if/else

Syntax:

```
if (condition1) {  
    //block of code to be executed if condition1 is true  
} else if (condition2) {  
    //block of code to be executed if the condition1 is false and condition2 is true  
}  
...  
else {  
    //block of code to be executed if the condition1 is false and condition2 is false  
}
```

◆ Condition Statements – if/else

```
var d = new Date();
var t = d.getHours();
document.write("<h4>" + d + "</h4>");
document.write("<h3>");
if (t < 12) {
    document.write("Good morning!"); }
else if (t < 18) {
    document.write("Good afternoon!"); }
else {
    document.write("Good evening!");
}
document.write("</h3>");
```



◆ Condition Statement – switch/case

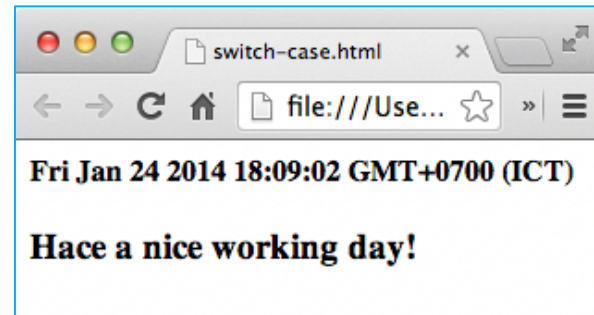
Syntax:

```
switch (expression) {  
    case x:  
        //code block to be executed when expression = x  
        break;  
    case y:  
        // code block to be executed when expression = y  
        break;  
    ...  
    default:  
        // code to be executed when all above cases not matched  
}  

```

◆ Condition Statement – switch/case

```
let d = new Date();
document.write("<h4>" + d + "</h4>");
document.write("<h3>");
switch (d.getDay()) {
  case 0:
  case 6:
    document.write("Have a lovely weekend!");
    break;
  default:
    document.write("Hace a nice working day!");
}
document.write("</h3>");
```



◆ Loop – for

Syntax:

```
for (statement 1; statement 2; statement 3) {  
    //code block to be executed  
}
```

Example:

```
for (let i = 1; i <= 6; i++) {  
    document.write("<h" + i + ">");  
    document.write("Heading " + i);  
    document.write("</h" + i + ">");  
}
```

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

◆ Loop – while, do/while

Syntax:

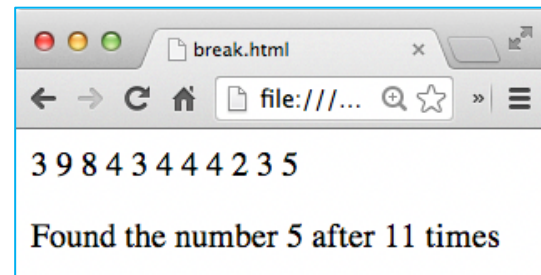
```
while (condition) {  
    // code block to be executed  
}
```

```
do {  
    //code block to be executed  
} while (condition);
```

◆ break statement

Used to jump out a loop or a switch statement and continue executing the code after the loop or switch

```
let count = 0;
while (true) {
  count++;
  let r = Math.floor(Math.random() * 10);
  document.write(r + " ");
  if (r == 5) {
    document.write("<p>Found '5' after "
      + count + " times</p>");
    break;
  }
}
```

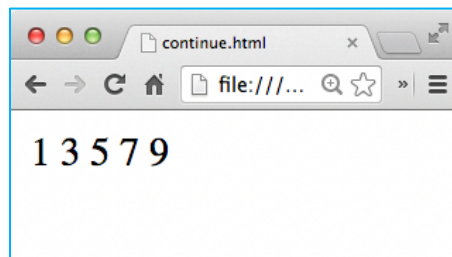


◆ continue statement

The continue statement jumps over an iteration of a loop

The condition of the loop is checked for a new iteration

```
for (let i = 0; i <= 10; i++) {  
  if (i % 2 == 0)  
    continue;  
  document.write(i + " ");  
}
```



Content

1. Introduction
2. Using JavaScript in an HTML documents
3. JS language basics
- 4. JS functions**
5. DOM
6. OOP in JS

◆ Function Declaration

Syntax:

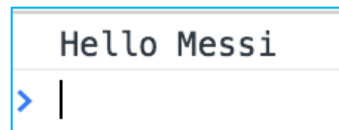
```
function func_name(parameters) {  
    //function body (statements)  
}
```

A function hasn't been executed until it is **called**

A **function call** can be made **before** its declaration (**hoisting**)

```
hello("Messi");
```

```
function hello(name) {  
    console.log("Hello " + name);  
}
```



◆ Function Expression

A JS function can also be defined using an **expression**

```
let x = function (a, b) {return a * b};  
let z = x(4, 3);  
alert(z); //alert "12"
```

The above function is actually an anonymous function (no name)

The function is referenced by the variable and it can **be called using the variable**

This also allow to use the **function as a parameter** or **return value** of a function

◆ Function Expression

```
function lessthan(a, b) { return a < b;}
function sort(lessthan, arr) { //pass function as a parameter
    for (i=0; i<arr.length-1; i++) {
        for (j=i+1; j<arr.length; j++) {
            if (lessthan(arr[j], arr[i]))
                swap(arr[i], arr[j]);
        }
    }
}
```

```
function d() {
    function e() {
        alert('E');
    }
    return e; //returns function e()
}
d(); //alerts 'E'
```

◆ Arrow Functions

Introduced in ES6

Allow us to write shorter function syntax:

Regular syntax and function expression:

```
function hello() {  
  return 'Hello';  
}
```

```
let hello = function  
( ) {  
  return 'Hello';  
}
```

Arrow function:

```
hello = ( ) => {  
  return 'Hello';  
}
```

```
hello = ( ) => 'Hello';
```

```
hello = (val) => 'Hello ' + val;
```

```
hello = val => 'Hello ' + val;
```

*If a function has **only one statement**, we can **remove the brackets** and the **return** keyword*

◆ Function Parameters

Function parameters (formal parameter) are the names **listed** in the function definition

Function arguments (actual parameter) are the real values **passed** to (and received by) the function

Parameter rules:

- JS function definitions do not specify **datatype** for parameters
- JS functions don't check the **datatype** of the passed arguments
- JS functions do not check the **number of arguments** received

The built-in object **arguments** contains an **array** of the arguments

◆ Function Parameters

```
x = findMax(1, 123, 500, 115, 44, 88);
```

```
function findMax() {  
    var i;  
    var max = -Infinity;  
    for (i = 0; i < arguments.length; i++) {  
        if (arguments[i] > max) {  
            max = arguments[i];  
        }  
    }  
    return max;  
}
```

◆ Pass-by-Reference & Pass-by-Value

Objects are pass by reference:

The change of the object properties are reflected outside the function

Arguments are pass by value:

The function only gets to know the value, not the argument's location

Change to arguments are not reflected outside the function

Content

1. Introduction
2. Using JavaScript in an HTML documents
3. JS language basics
4. JS functions
- 5. DOM**
6. OOP in JS

◆ What is HTML DOM?

The **DOM** is a **W3C standard** for accessing documents, including core, XML and HTML DOM

The **HTML DOM** is a standard **object model** and **programming interface** for HTML, which defines:

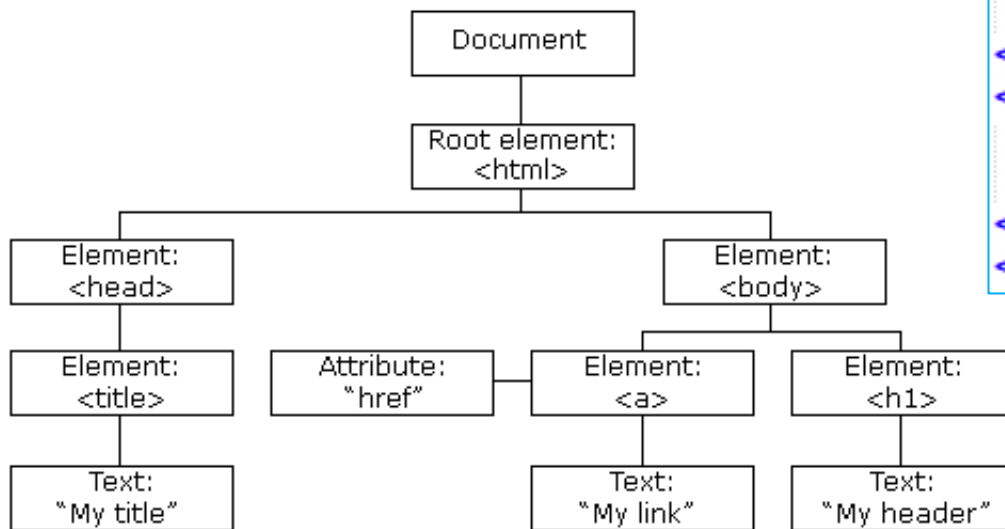
- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

HTML DOM: a standard for getting, changing, adding, or deleting HTML elements

HTML DOM Model

The HTML DOM model is constructed as a tree of objects

When a web page is loaded, the browser creates a Document Object Model of the page



```
<html>
<head>
  <title>My title</title>
</head>
<body>
  <a href="...">My link</a>
  <h1>My header</h1>
</body>
</html>
```

◆ HTML DOM Model

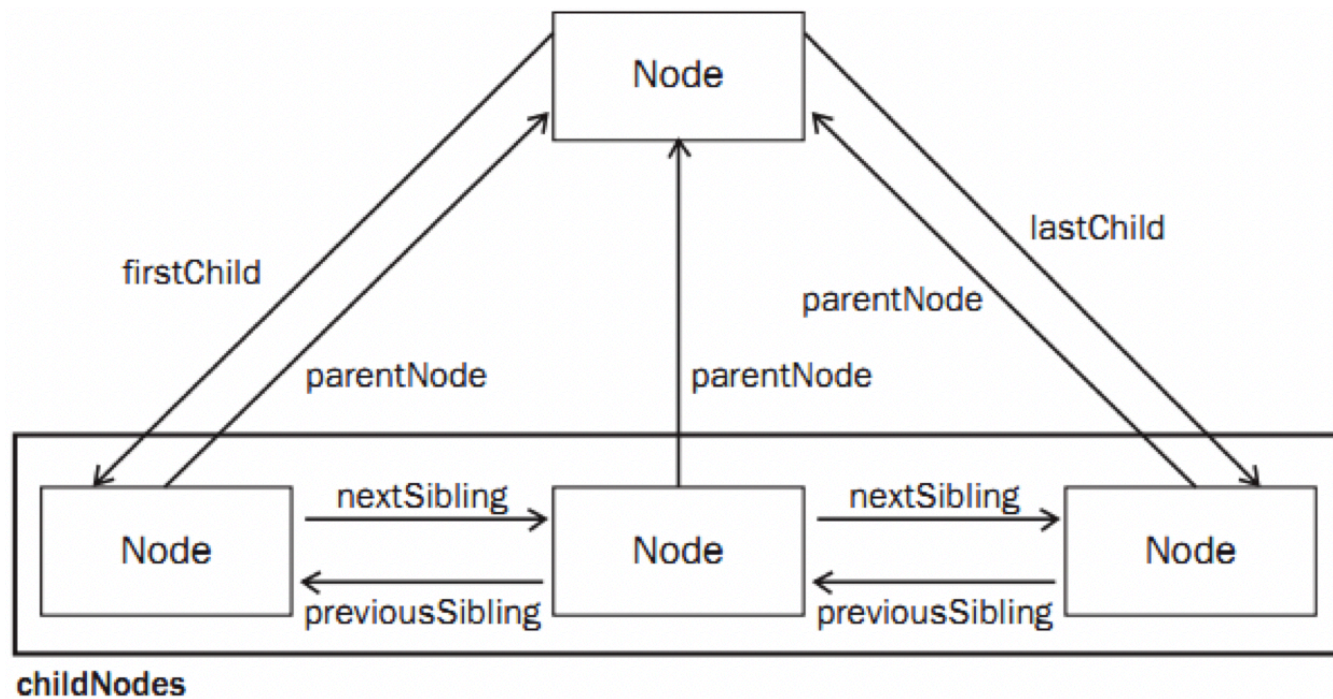
The objects in the hierarchical tree are called **nodes**

Basic **node types**:

- **Document node** (Node.**DOCUMENT_NODE, 9**): represents the entire document (the root-node of the DOM tree, children: Elements)
- **Element node** (Node.**ELEMENT_NODE, 1**): represents an element (children: Elements, Texts, Comments)
- **Text node** (Node.**TEXT_NODE, 3**): represents textual content in an element or attribute (children: none)
- **Attribute node** (Node.**ATTRIBUTE_NODE, 2**): represents an attribute (children: none)

HTML DOM Model

Document Object Model (DOM)



◆ HTML DOM Methods

A method is an **action** that we can perform on **HTML elements**

Methods	Description
Selecting HTML element	
<code>document.getElementById(id)</code>	Find an element by element id
<code>document.getElementsByTagName(tag)</code>	Find elements by tag name
<code>document.getElementsByClassName(class)</code>	Find elements by class name
<code>document.querySelector(css selectors)</code>	Returns the first child element that matches the given CSS selector(s)
<code>document.querySelectorAll(css selectors)</code>	Returns all elements that matches the given CSS selector(s)

◆ HTML DOM Methods

A method is an **action** that we can perform on **HTML elements**

Methods	Description
Adding and Deleting elements	
<code>document.createElement(element)</code>	Create an HTML element
<code>document.createTextNode(text)</code>	Create a text node
<code>parentNode.appendChild(node)</code>	Add an HTML element
<code>parentNode.removeChild(node)</code>	Remove an HTML element
<code>parentNode.replaceChild(new, old)</code>	Replace an HTML element
<code>document.write(text)</code>	Write into the HTML output stream

◆ HTML DOM Properties

Properties: values (of HTML elements) that we can **set** or **change**

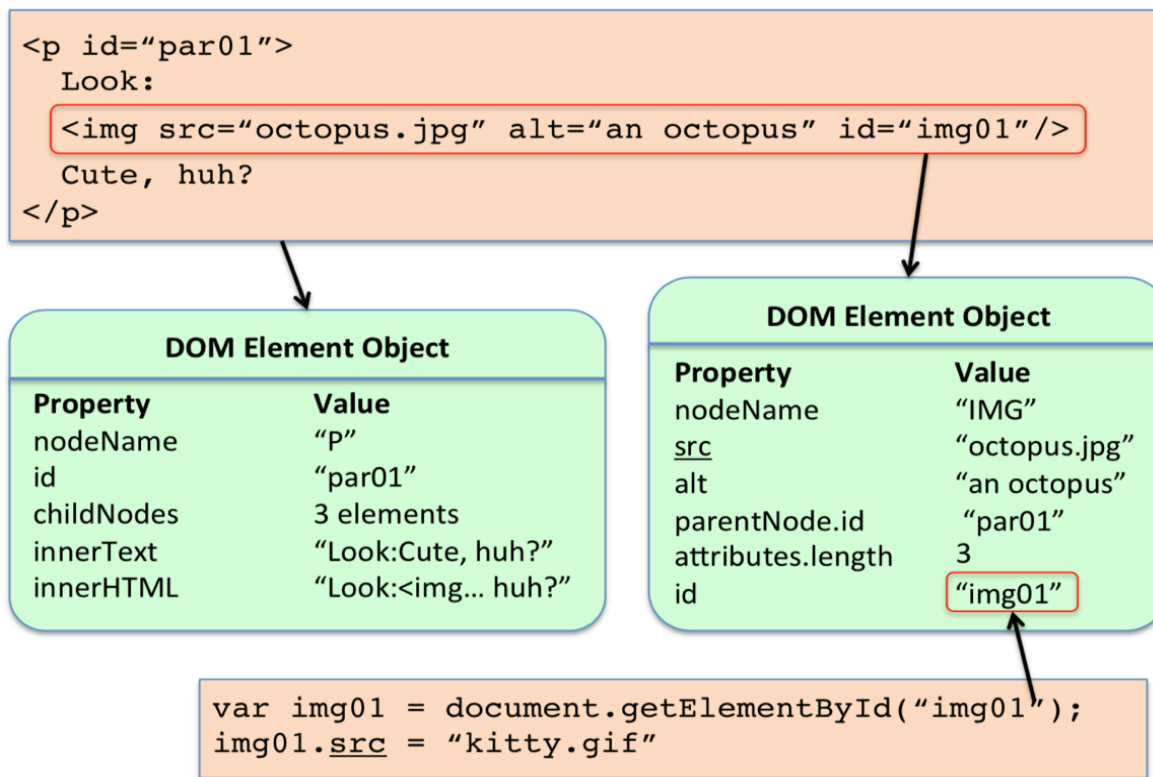
Each type of node has its own **set of attributes**

- **innerText**: the text content of the specified node (and all its descendants)
- **innerHTML**: the HTML content of the specified node
- **nodeValue**: the node value of the specified node
- **nodeName**: the name of the specified node
- **parentNode**: the parent of the specified node (Node object)
- **childNodes**: the child nodes (NodeList object)
- **attributes**: attributes of the specified node (NamedNodeMap obj.)

Full list: https://www.w3schools.com/jsref/dom_obj_all.asp

HTML DOM Properties

Document Object Model (DOM)



◆ HTML DOM Properties

nodeName:

- Element node: tag name (e.g. “P”, “IMG”, etc.)
- Attribute node: attribute name (e.g. “src”, ”href”, etc.)
- Text/Document node: “#text”, “#document”

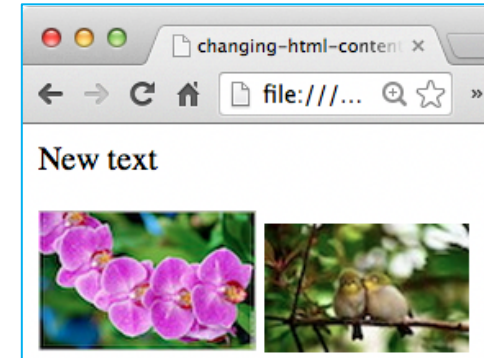
nodeValue:

- Element node: “undefine”
- Attribute node: value of the attribute
- Text node: text value of the node

Note: to access a property value, using the syntax
`<node>.<attribute_name>`

◆ Change Element Content and Attribute

```
<html>
  <body>
    <p id="par1">Hello World</p>
    <p>
      
    </p>
    <script>
      let p1 = document.getElementById("par1");
      p1.innerHTML = "New text";
      let m2 = document.getElementById("img2");
      m2.src = "birds.jpg";
    </script>
  </body>
</html>
```



◆ Change Element Style

Syntax: `<node>.style.<property> = <value>`

```
<body>
```

```
<p id="par1">The first paragraph</p>
```

```
<p id="par2">The second paragraph</p>
```

```
<script>
```

```
let p1 = document.getElementById("par1");
```

```
p1.style.fontSize = "x-large";
```

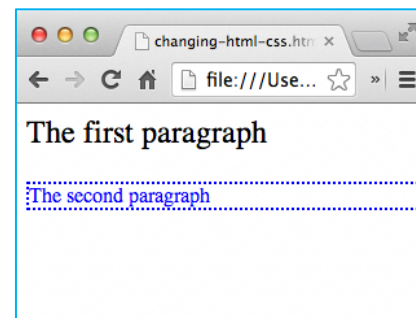
```
let p2 = document.getElementById("par2");
```

```
p2.style.color = "blue";
```

```
p2.style.border = "2px dotted #0000FF";
```

```
</script>
```

```
</body>
```



◆ Change Element Class

Syntax: `<node>.className = <classname>`

```
<head>
```

```
  <style>
```

```
    .blue {color: blue; border: 2px dotted #0000FF;}
```

```
    .big {font-size: x-large;}
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <p id="par1">The first paragraph</p>
```

```
  <p id="par2">The second paragraph</p>
```

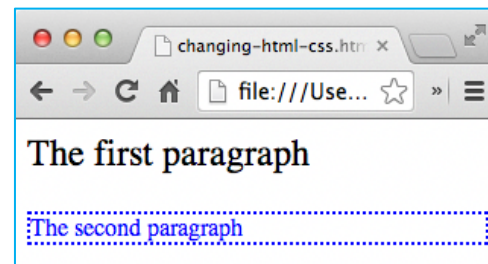
```
  <script>
```

```
    document.getElementById("par1").className = "big";
```

```
    document.getElementById("par2").className = "blue";
```

```
  </script>
```

```
</body>
```



◆ Creating new HTML element

```
<body>
```

```
  <div id="div1">
```

```
    <p id="par1">The first paragraph</p>
```

```
    <p id="par2">The second paragraph</p>
```

```
  </div>
```

```
  <script>
```

```
    const p1 = document.createElement("p");
```

```
    const node = document.createTextNode("This is new.");
```

```
    p1.appendChild(node);
```

```
    const element = document.getElementById("div1");
```

```
    element.appendChild(p1);
```

```
    // element.insertBefore(newChild, referenceChild);
```

```
  </script>
```

The first paragraph

The second paragraph

This is new.

◆ Remove HTML element

```
<body>
  <div id="div1">
    <p id="par1">The first paragraph</p>
    <p id="par2">The second paragraph</p>
  </div>
  <script>
    const parent = document.getElementById("div1");
    const element = document.getElementById("par1");
    element.remove();
    // parent.removeChild(element)
  </script>
</body>
```

◆ Replacing HTML element

```
<body>
```

```
  <div id="div1">
```

```
    <p id="par1">The first paragraph</p>
```

This is new.

```
    <p id="par2">The second paragraph</p>
```

The second paragraph

```
  </div>
```

```
  <script>
```

```
    const p1 = document.createElement("p");
```

```
    const node = document.createTextNode("This is new.");
```

```
    p1.appendChild(node);
```

```
    const parent = document.getElementById("div1");
```

```
    const element = document.getElementById("par1");
```

```
    parent.replaceChild(p1, element);
```

```
  </script>
```

```
</body>
```

◆ HTML Events

HTML provides event handler attributes to use JS code to handle the events

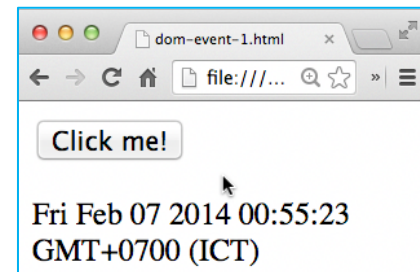
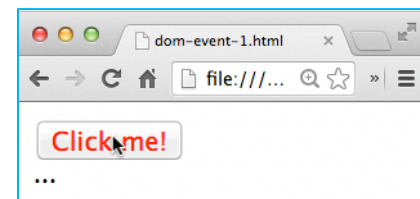
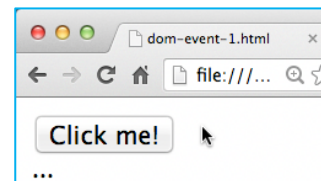
```
<element event='some JS code'>
```

Some common HTML events:

- Mouse: onclick, ondblclick, onmousedown, onmousemove, onmouseover, onmouseout, onmouseup
- Keyboard: onkeypress, onkeyup, onkeydown
- Form: onblur, onchange, onfocus, onsubmit, onselect
- Frame/Object: onload, onresize, onscroll

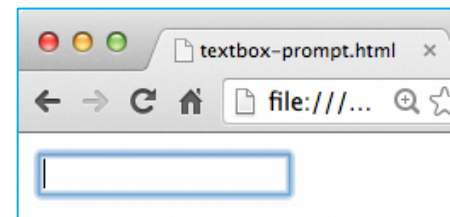
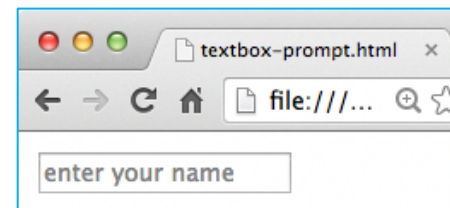
HTML Events

```
<html>
  <head>
    <script>
      function displayDate(eid) {
        let e = document.getElementById(eid);
        e.innerText = Date();
      }
    </script>
  </head>
  <body>
    <input type="button" value="Click me!" id="bt1"
      onclick="displayDate('demo');"
      onmouseover="this.style.color='red';"
      onmouseout="this.style.color='black';"/>
    <p id="demo">...</p>
  </body>
</html>
```



◆ HTML Events

```
<script>
function prompttext(getf, inp) {
    if (getf && (inp.value == "enter your name")) {
        inp.value = "";
        inp.style.color = "black";
    }
    if (!getf && (inp.value == "")) {
        inp.value = "enter your name";
        inp.style.color = "gray";
    }
}
</script>
```



```
<input id="uname" type="text" style="color:gray;"
value="enter your name" onfocus="prompttext(true, this);"
onblur="prompttext(false, this);"/>
```

◆ DOM Event Listeners

Each DOM object has the following method for **event handling**:

```
addEventListener(eventName, functionName);
```

event name: the string of name of the event to listen to

function name: the name of the JS function that will be executed when the event fires (this function gets an **Event object** as param)

Pros: **separate** JS code from HTML markup
An event can be handled by **multiple functions**

To **stop** listen to an event use the following function:

```
removeEventListener(eventName, functionName);
```

(the parameters are similar to the `addEventListener()`)

HTML DOM Listeners

```
<script>
  document.getElementById("myButton").addEventListener("click",
validate);
  function validate(event) {
    alert("You click the object: " + event.target);
    var x = document.getElementById("fname").value;
    if (x == "")
      alert("Name must be filled out");
    else
      alert("Hello: " + x);
  }
</script>
```

Name

An embedded page on this page says

Hello: Jerry

```
<body>
  <p> Name <input type="text" name="fname" id="fname">
  <input id="myButton" type="button" value="Click me!">
  </p>
</body>
```

https://www.w3schools.com/js/js_events.asp

Content

1. Introduction
2. Using JavaScript in an HTML documents
3. JS language basics
4. JS functions
5. DOM
- 6. OOP in JS**

◆ JS Objects

In JS, almost "everything" is an object, except primitives

- **Boolean can be objects** (if defined with the new keyword)
- **Number can be objects** (if defined with the new keyword)
- **String can be objects** (if defined with the new keyword)
- **Date are always objects**
- **Math are always objects**
- **Regular expressions are always objects**
- **Arrays are always objects**
- **Functions are always objects**
- **Objects are always objects**

◆ JS Objects

A variable can contain a single value

JS objects:

- A JS object can contain many **named values (properties)**
- The values are written in pair as **name : value**
- An object can also contain **methods** (actions that the object can performed)
- A method can be considered as a property that contains a function definition

◆ Create a JS Object

Three ways to create an object:

Define and create a single object, using an **object literal**

```
var person = {firstName:"John", lastName:"Doe", age:50,  
  showInfo: function() {  
    alert(this.name);  
  }  
};
```

Define and create a single object with the **keyword new**

```
var person = new Object();  
person.firstName = "John";  
person.age = 50;  
person.showInfo = function() { alert(this.name); };
```

◆ Create a JS Object

Three ways to create an object (cont):

Define an **object constructor function**, and then create objects of the constructed type

```
function User(uname, pass) {  
    this.uname = uname;  
    this.pass = pass;  
    this.showInfo = function() {  
        alert(this.uname + ":" + this.pass);  
    } //showInfo()  
} //User
```

```
var alibaba = new User("alibaba", "vomc");  
alibaba.showInfo(); //alert "alibaba:vomc"
```

◆ Add Object Properties

To add a property to an object, name the property and give it a value:

```
const scores = {  
  peach: 100,  
  mario: 88,  
  luigi: 91  
};  
scores.toad = 72;  
let name = 'wario';  
scores[name] = 102;  
console.log(scores);
```

► *Object {peach: 100, mario: 88, luigi: 91, toad: 72, wario: 102}*

◆ Remove Object Properties

To remove a property of an object, use delete:

```
const scores = {  
  peach: 100,  
  mario: 88,  
  luigi: 91  
};  
scores.toad = 72;  
let name = 'wario';  
scores[name] = 102;  
delete scores.peach;  
console.log(scores);
```

```
► Object {mario: 88, luigi: 91, toad: 72, wario: 102}
```

◆ Iterate through Object Properties

Use the **for ... in** loop (for each property in the object):

```
for (key in object) {  
    //do something with object[key]  
}
```

Example:

```
for (let name in scores) {  
    console.log(name + ' got ' + scores[name]);  
}
```

```
mario got 88
```

```
luigi got 91
```

```
toad got 72
```

```
wario got 102
```


◆ Prototype

JS is described as a **prototype-based language**

Prototype may be considered as a “class” of an object created by an object constructor function

A prototype is created for every object constructor function

It is used to:

- **create static** (common) **properties** for objects
- **add properties** (inherit) to an existing object created by an object constructor function

◆ Prototype

```
function Person(first, last) {
  this.first = first;
  this.last = last;
}
var person1 = new Person("John", "Doe");
```



To add a method for Person:

```
Person.prototype.showInfo = function() {
  alert(this.first + " " + this.last);
};
```

```
person1.showInfo();
```

◆ Prototype

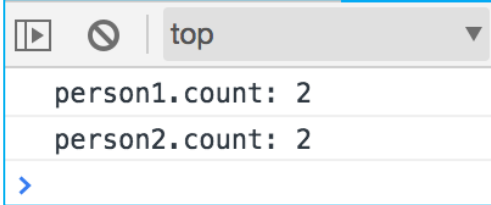
Add a “static” property for a prototype:

```
function Person(first, last) {
  this.firstName = first;
  this.lastName = last;
  Person.prototype.count++;
};
```

```
Person.prototype.count = 0;
```

```
var person1 = new Person("John", "Doe");
var person2 = new Person("Doe ", "John");
```

```
console.log("person1.count: " + person1.count);
console.log("person2.count: " + person1.count);
```



person1.count: 2
person2.count: 2

JS Classes

Creating classes in ES6:

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name}
      makes a sound`);
  }
}
```

```
class Dog extends Animal {
  constructor(name, breed) {
    //Call the constructor of the parent class
    super(name);
    this.breed = breed;
  }

  //Overriding the speak method of the parent class
  speak() {
    console.log(`${this.name} barks`);
  }

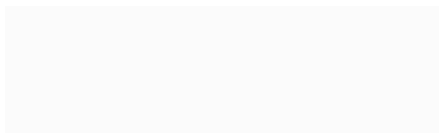
  //New method specific to the Dog class
  fetch() {
    console.log(`${this.name}
      fetches a ball`);
  }
}
```

◆ Creating Objects

Creating objects in ES6:

```
//Creating an instance of the child class  
let myDog = new Dog('Buddy', 'Golden Retriever');  
  
//Accessing properties and methods of the child class  
console.log(myDog.name);           //Output: Buddy  
console.log(myDog.breed);          //Output: Golden Retriever  
myDog.speak();                     //Output: Buddy barks  
myDog.fetch();                     //Output: Buddy fetches a ball
```

Appendix



◆ Alert Box

Used to make sure information comes through to the user

The user will have to click "OK" to proceed

Syntax: `[window.]alert("sometext");`

An embedded page on this page says

I am an alert box!

OK

◆ Confirm Box

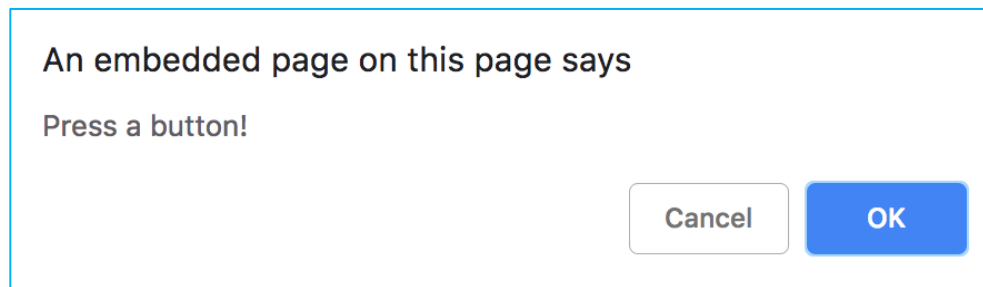
Used verify that the user accept something or not

The user will have to click either "OK" or "Cancel" to proceed

If the user clicks "OK", the box returns **true**.

If the user clicks "Cancel", the box returns **false**

Syntax: `[window.]confirm("sometext");`



◆ Prompt Box

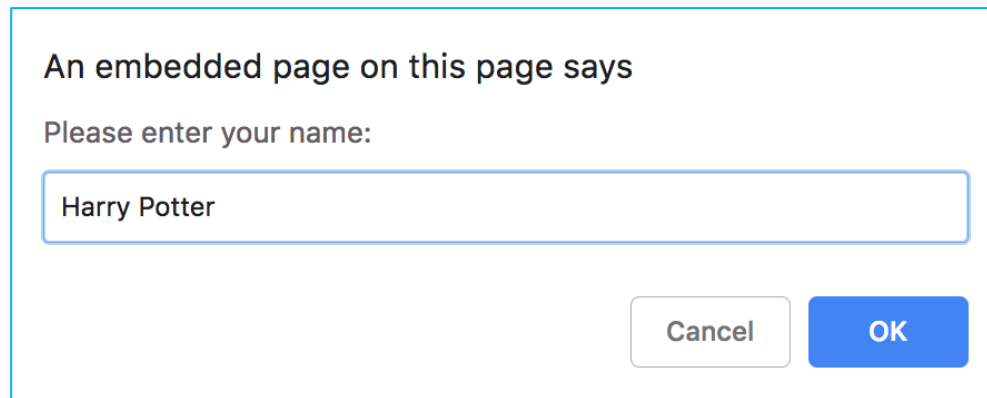
Used to get an input value from users

The user will have to click either "OK" or "Cancel" to proceed

If the user clicks "OK" the box returns the input value.

If the user clicks "Cancel" the box returns **null**

Syntax: `[window.]prompt("sometext", "defaultText");`



An embedded page on this page says

Please enter your name:

Harry Potter

Cancel OK

◆ Exception

try ... catch is used to catch an exception occurs

Syntax:

```
try {  
    Block of code to try  
}  
catch(err) {  
    Block of code to handle errors  
}  
finally {  
    Block of code to be executed regardless of the try / catch result  
}
```

Finally is optional

◆ Exception

Example:

```
var txt="";
function message()
{
  try
  {
    adddlerter("Welcome guest!");
  }
  catch(err)
  {
    txt="There was an error on this page.\n\n";
    txt+="Error description: " + err.description + "\n\n";
    txt+="Click OK to continue.\n\n";
    alert(txt);
  }
}
```

◆ Further Reading

Methods of JS basic datatypes:

- String: indexOf(), lastIndexOf(), slice(), split(), trim(), etc.
- Number: isFinite(), isInteger(), toString(), etc.
- Math: abs(), min(), max(), sqrt(), round(), random(), etc.

JS regular expression: https://www.w3schools.com/js/js_regexp.asp

JS debugging: https://www.w3schools.com/js/js_debugging.asp

JS style guide: https://www.w3schools.com/js/js_conventions.asp

JS Browser Object Model (BOM, allows JS to "talk to" the browser): Window, Screen, Location, History, Navigation, Cookies



Question?

Chapter 4 – JavaScript

◆ Exercise 1: Show/Hide Button

- Toggle visibility of an element with a button
- Add button: `<button id="toggleBtn">Show Details</button>`
- Add content:
`<div id="details" style="display: none;">Hidden content here</div>`

- JavaScript:

```
const btn = document.getElementById('toggleBtn');
const details = document.getElementById('details');
btn.addEventListener('click', function() {
    if (details.style.display === 'none') {
        details.style.display = 'block';
        btn.textContent = 'Hide Details';
    } else {
        details.style.display = 'none';
        btn.textContent = 'Show Details';
    }
});
```

◆ Exercise 2: Change Background Color

- Button that changes page background color randomly
- Add HTML: <button id="colorBtn">Change Color</button>
- JavaScript:

```
const colorBtn = document.getElementById('colorBtn');
colorBtn.addEventListener('click', function() {
    const colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A',
    '#98D8C8', '#F7DC6F'];
    const randomColor = colors[Math.floor(Math.random() *
    colors.length)];
    document.body.style.backgroundColor = randomColor;
});
```

◆ Exercise 3: Character Counter

- Count characters as user types in textarea

- Add HTML:

```
<textarea id="myText" maxlength="200"></textarea>  
<p><span id="charCount">0</span> / 200 characters</p>
```

- JavaScript:

```
const textarea = document.getElementById('myText');  
const charCount = document.getElementById('charCount');  
textarea.addEventListener('input', function() {  
    charCount.textContent = textarea.value.length;  
});
```


◆ Exercise 4: Image Gallery with Prev/Next

- Navigate through images with buttons
- Add HTML:

```
<button id="prevBtn">← Previous</button>
```

```

```

```
<button id="nextBtn">Next →</button>
```

◆ Exercise 4: Image Gallery with Prev/Next

- JavaScript:

```
const images = ['image1.jpg', 'image2.jpg', 'image3.jpg', 'image4.jpg'];
let currentIndex = 0;
const img = document.getElementById('galleryImg');
const prevBtn = document.getElementById('prevBtn');
const nextBtn = document.getElementById('nextBtn');
nextBtn.addEventListener('click', function() {
    currentIndex = (currentIndex + 1) % images.length;
    img.src = images[currentIndex];
});
prevBtn.addEventListener('click', function() {
    currentIndex = (currentIndex - 1 + images.length) % images.length;
    img.src = images[currentIndex];
});
```

◆ Exercise 5: Countdown

- Count down from a number to 0

- Add HTML

`<p id="timer">10</p> <button id="startBtn">Start</button>`

- JavaScript

Exercise 5: Countdown

```
let timeLeft = 10;
let timerInterval=null;
const timerDisplay = document.getElementById('timer');
const startBtn = document.getElementById('startBtn');
startBtn.addEventListener('click', function() {
    if (timerInterval !== null) return;
    timeLeft = 10;
    timerInterval = setInterval(function() {
        timeLeft--;
        timerDisplay.textContent = timeLeft;
        if (timeLeft <= 0) {
            clearInterval(timerInterval);
            timerInterval = null;
            alert('Time is up!');
            timerDisplay.textContent = 10;
        }
    }, 1000);
});
```

◆ Exercise 6: Typing effect

- Text appears letter by letter like typing
- Add HTML: `<p id="typeText"></p>`
- JavaScript

```
const text = "Welcome! Learn JavaScript step by step.";
let index = 0;
function typeWriter() {
    if (index < text.length) {
        document.getElementById('typeText').textContent +=
text.charAt(index);
        index++;
        setTimeout(typeWriter, 100);
    }
}
typeWriter();
```